

Recompilation Is Not Enough: Test-Guided Decompile-C Repair

Yuhan Huang
Xidian University
Xi'an, China
Ant Group
Hangzhou, China

Puzhuo Liu
Ant Group
Hangzhou, China

Jianlei Chi
Xidian University
Xi'an, China

Abstract

Decompiled C often becomes recompilable only after repair, but recompilation alone does not establish test-observed behavior. A recompiled command-line binary can still parse options incorrectly, print different bytes, or return a different exit status. We present a few-step workflow for repairing decompiled C using compiler feedback and related official tests. Compiler and linker diagnostics first guide build repair. Once the repaired C recompiles into a binary, smoke checks and related official tests expose behavioral discrepancies for semantic repair. In a preliminary static-enriched evaluation on 104 Coreutils 9.5 binaries with available decompiler exports and deterministic exact-output smoke comparisons, 91 binaries (87.5%) recompile and pass the test gate; 9 do not recompile within the repair budget, and 4 recompile but still fail the test gate. The result suggests that test-gate feedback can make LLM-assisted repair of decompiled C more auditable than compile-only recovery.

Keywords

decompiled C repair, recompilation, program repair, large language models, software testing

1 Introduction

Decompilers can generate seemingly plausible C code for reverse engineering or taint analysis[11, 12]. The decompiled result can even be recompiled, but the resulting binary may still parse options differently, print different bytes, or return different exit statuses. This paper studies that gap for complete command-line binaries. Recent work has made LLM-based binary decompilation more concrete: assembly-language models and large decompilation benchmarks broaden the training and evaluation setting [7, 17], two-phase decompilation separates structural recovery from identifier recovery [15], and refinement systems target recompilable output, distorted pseudocode, or decompilation fidelity [8, 19, 23]. For decompiled C, the remaining repair gap is behavioral: a recompiled binary can still diverge from the original utility’s command-line behavior.

The gap is sharper when the repair unit is a complete binary rather than an individual function. Function-level decompilation settings can evaluate recovered functions in controlled harnesses [1, 17], but a repaired Coreutils binary must recover option tables, static data, global state, helper-library behavior, output formatting, and exit-status conventions. Recent evaluation work separates readability, recompilability, and functionality [10], while equivalence checking, fidelity studies, type recovery, and broader binary-analysis benchmarks show that recovered C and binary-analysis outputs can fail along different axes [3, 4, 14, 18]. We therefore treat

recompilation as an intermediate step and use related official tests as repair feedback.

The workflow repairs one decompiled Coreutils binary at a time. It first uses compiler and linker diagnostics to repair the decompiled C until it recompiles. It then validates behavior with smoke checks and related official tests. We use related official tests to mean Coreutils tests drawn from the official suite and assigned before repair because they exercise the target binary’s expected command-line behavior. They are not Coreutils source files or complete answer files; compact failure summaries may include observable discrepancy fragments. Test failures are summarized into repair evidence, and the next repair step uses those discrepancies to modify the decompiled C. We do not use Coreutils source code as repair input.

This setting differs from both decompiler readability evaluation and source-level automated repair. Decompiled C may contain type reconstruction errors, missing global data, synthetic control flow, and library-boundary confusion at the same time. The repair target is also not a localized regression in a maintained program; it is a recompiled binary whose observable command-line behavior should match the original utility under the test gate. For that reason, the workflow is deliberately conservative: a binary is accepted only if the repaired C recompiles and the recompiled binary passes related official tests and smoke checks.

This work-in-progress paper makes two contributions: (1) a few-step workflow for repairing decompiled C at binary granularity, and (2) a test-gate feedback loop that guides semantic repair beyond recompilation.

As preliminary evidence, we evaluate 104 GNU Coreutils 9.5 binaries in the static-enriched track, restricted to binaries with available decompiler exports and deterministic exact-output smoke comparisons. Under a bounded few-step policy with a primary three-iteration build target, recorded diagnostic extensions, and a three-iteration semantic cap, 91 of the 104 binaries (87.5%) recompile and pass the test gate; 9 do not recompile within the repair budget, and 4 recompile but still fail the test gate.

2 Motivation

Recompilation exposes build-level faults, but it does not establish behavior. Compiler diagnostics identify missing declarations, type conflicts, and unresolved symbols; they rarely explain why an option combination should produce a specific byte sequence or exit status. Smoke checks are useful sanity checks, but related official tests can expose boundary cases, option interactions, and user-visible conventions accumulated by the utility maintainers.

The distinction also determines which repair evidence is useful. A compiler error usually points to a local syntactic defect, while a behavioral failure often reflects a cross-function invariant: a global option table may be malformed, a recovered helper may have the

Figure 1 shows the evidence gap. Compiler and linker diagnostics establish that the repaired C can produce a binary, and smoke checks establish that a shallow command-line path works. In the cut case, however, the recompiled binary passed only 2 of 5 related official tests, with output, exit-status, and help/version discrepancies. The case is small, but it illustrates the paper’s central point: related official tests expose repair evidence that compiler diagnostics and smoke checks do not capture.

Every repair attempt records recompilation and test-gate status. Build repair has a primary three-iteration target with recorded

diagnostic extensions, semantic repair is capped at three iterations, and failures remain explicit.

Build repair. The first stage turns decompiler output into C that can be compiled and linked. Deterministic preprocessing handles recurring, source-independent artifacts: declarations and headers, decompiler intrinsic helpers, ABI and prototype cleanup, recovered global data declarations, and minimal C/POSIX helper shims. These fixes are justified by C/POSIX semantics, ABI conventions, binary-visible symbols, static metadata, or black-box original-binary behavior. If preprocessing does not produce a recompiled binary, LLM build repair receives the decompiled C and compiler/linker diagnostics. The primary build-repair target is three iterations; any bounded diagnostic extension is recorded as repair effort rather than hidden from the final accounting.

The deterministic rules are intentionally narrow. They normalize artifacts that appear repeatedly in decompiled C, such as inconsistent integer typedefs, missing `libc` prototypes, synthetic temporaries, or recovered data objects that lack declarations. A rule is acceptable only when its justification is independent of a particular test outcome. This keeps preprocessing closer to a reusable post-decompilation repair layer and reduces the risk that it becomes a hidden benchmark-specific solution.

Test gate. Recompilation is not enough. After the repaired C recompiles, the workflow executes the recompiled binary and validates behavior with related official tests plus smoke checks. A binary passes the test gate only when it recompiles and passes both forms of behavioral evidence. Separately, generated repairs are audited against known test-specific branch patterns and hard-coded answer patterns. Unlike function-level decompilation settings, this gate executes the repaired binary as a standalone command-line program and exposes missing cross-function or global behavior [1, 17]. Test evidence is assigned before repair, following the observation that repair models benefit from relevant facts but can degrade when supplied with indiscriminate context [13].

Smoke checks and related official tests play different roles. Smoke checks are cheap and deterministic, so they catch severe regressions such as a binary that cannot start or mishandles a canonical command. Related official tests are more targeted and can expose boundary ranges, option interactions, diagnostic formatting, and error-path behavior that simple smoke inputs often miss. Requiring both checks prevents the workflow from accepting a binary that merely satisfies a shallow command-line trace.

Semantic repair. When tests fail, the workflow summarizes what differed: failing tests, stdout/stderr mismatch summaries, exit-status mismatches, timeouts, and smoke-check results. Deterministic repair handles recurring defect classes such as malformed static data, recovered option tables, incorrect helper signatures, data-layout constants, and error-path conventions. These recoveries are broad categories, not complete answer files. Remaining cases are passed to LLM semantic repair with the compact discrepancy summary. Each semantic edit is rebuilt before retesting; if the edit introduces a compile or link failure, the resulting diagnostics are included in the next feedback summary for that semantic attempt. After three semantic-repair iterations, binaries that still fail the test gate are counted as failures. This bounded loop is related to

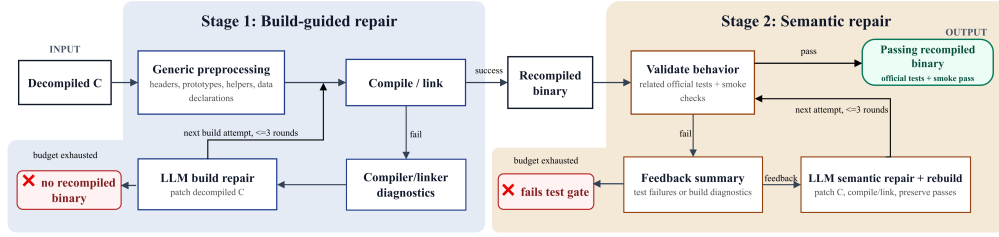


Figure 2: Bounded two-stage repair workflow. Compiler/linker feedback guides build repair; related official tests and smoke checks guide semantic repair.

recent LLM repair agents that iterate over execution and test feedback [2, 22], but the starting point here is a compound decompiler artifact rather than a localized source-level bug.

The discrepancy summary is compact by design. It does not forward complete test logs or complete expected-output files as repair targets. Test identifiers and short discrepancy fragments may be retained for audit and context, while generated repairs are audited against known test-name branches, test-specific command-line branches, Coreutils source snippets, and answer-key branches. The summary emphasizes observable discrepancies and asks the repair step to modify the underlying utility logic while preserving already passing behavior. This is a practical compromise between giving too little evidence, which leads to blind edits, and giving unrestricted transcripts, which increases overfitting risk.

4 Evaluation

4.1 Setup

The preliminary evaluation uses 104 GNU Coreutils 9.5 binaries with available decompiler exports and deterministic exact-output smoke comparisons. All results use the static-enriched track: repair may use sanitized binary/static-analysis sidecars, including recovered symbols, constants, and data-layout hints, but excludes Coreutils source snippets and complete expected-output files.

The unit of accounting is the binary, not an individual function. A binary is accepted only when the repaired C recompiles, links, and the recompiled binary passes the test gate: related official tests plus smoke checks. Related official tests are Coreutils tests drawn from the official suite and assigned before repair because they exercise the target binary’s expected command-line behavior. They provide behavioral evidence without providing Coreutils source files or complete answer files as repair input. Full Coreutils suite results, when collected, are stress/audit artifacts rather than the primary completion result.

Each binary is repaired under a bounded few-step policy: build repair has a primary three-iteration target with recorded diagnostic extensions, and semantic repair is capped at three iterations. Candidates that do not compile and link within the build budget are build failures; candidates that recompile but still fail the test gate after semantic repair are behavioral failures. The budget is part of the method because it keeps successes, extensions, and failures visible under the same accounting rule.

Table 1: Outcomes for 104 Coreutils 9.5 binaries under bounded repair.

Outcome under bounded repair	Count
Pass test gate	91
Do not recompile within the repair budget	9
Recompile but still fail test gate	4
Total evaluated	104

4.2 Effectiveness

Each binary is tracked as an end-to-end repair artifact with repair history, recompilation status, and test-gate status. This matters because repaired C alone can be misleading: it can recompile, pass smoke checks, and still fail related official tests. The evaluation therefore reports passing recompiled binaries and explicit failure stages for the rest.

Table 1 reports the final outcome taxonomy. In the static-enriched track, 91 of the 104 evaluated binaries (87.5%) recompile and pass the test gate. The 13 remaining cases are preserved as explicit failures under the same bounded budget: 9 do not recompile within the build-repair budget, while 4 recompile but still fail the test gate after semantic repair.

This taxonomy separates two failure modes that would be collapsed by a compile-only metric. The 9 build failures point to unresolved post-decompilation C integration problems, such as declarations, recovered data, cross-file linkage, or helper behavior needed to produce a standalone command-line binary. The 4 test-gate failures are recompiled binaries whose observable behavior still diverges under related official tests or smoke checks.

The cut case in Figure 1 illustrates the evidence gap that triggers semantic repair. The recompiled binary passed smoke checks, but related official tests exposed the remaining behavior gap. In the full repair trace, two semantic-repair iterations then produced a binary that passed the related official tests for this case with no smoke regression. The discrepancy summary captured observable mismatches that point to user-visible behavior, while the repair checks audited the edit against known test-name branches and other test-specific shortcut patterns.

The trace also shows why semantic repair is not just another build-repair pass. The related official tests covered ordinary data processing and command-line conventions, including range parsing, read/write error paths, and help/version handling. The first

recompiled binary had enough structure to run, but the discrepancy summary exposed behavioral categories that compiler diagnostics could not explain.

4.3 Efficiency

Across the 104-binary scope, the logs record 6.87M tokens, computed by summing build-repair trace tokens and semantic-repair tokens across successes, build-budget failures, diagnostic extensions, and test-gate failures. Of 95 binaries entering the test gate, 91 passed and 4 failed after the bounded semantic loop; the other 9 are build-stage failures. These records profile prototype repair effort rather than model-independent runtime efficiency, since wall-clock time depends on runner load, test scheduling, and external service latency.

5 Discussion

The preliminary result supports a repair discipline rather than a final decompilation benchmark. Under one bounded budget, binaries that do not recompile remain build failures, and recompiled binaries that fail the test gate are not accepted on smoke checks alone. This accounting limits the extent to which unbounded retry or manual steering can obscure the source of progress.

The current evaluation remains narrow. It covers one utility family, related official tests rather than full equivalence, and one primary static-enriched track. The deterministic layer is evidence-backed, but its portability beyond Coreutils-style binaries remains an open question. The exact-output smoke checks also favor deterministic command-line behavior. A clean pseudo-only ablation, more detailed component ablations, broader utility families, and root-cause analysis by failure family are future work.

There are also measurement threats. Related official tests are stronger than smoke checks, but they are still a subset of full behavioral equivalence. Static metadata may make some repairs easier than a pseudo-only setting, so future experiments should isolate the value of metadata, deterministic normalization, LLM build repair, and LLM semantic repair. Finally, transferring the workflow to libraries, servers, or interactive programs will require different behavioral tests and different rules for defining an externally observable contract.

6 Related Work

Recent LLM-based decompilation. Recent systems move beyond lexical similarity toward executability and richer binary context. Nova and Decompile-Bench strengthen assembly-language modeling and large-scale evaluation [7, 17], while SK2Decompile separates source-structure recovery from identifier recovery [15]. PseudoFix targets distorted decompiled C pseudocode [8]. Earlier formally published systems such as LLM4Decompile and SLaDe provide useful foundations for decompilation generation and optimized assembly decompilation [1, 16]; ReSym, SymGen, and GenNm show how LLMs can recover symbols and names from stripped binaries [6, 20, 21]. These systems improve recovered function quality, but they do not by themselves define when a complete recompiled binary should be accepted as passing a test gate.

Repair and correction loops. Decompiler-output refinement increasingly uses recompilation, correction, or validation feedback.

DecLLM [19] and FirmNamer [9] refine decompiler output for program understanding; FidelityGPT uses retrieval-augmented correction [23], and DeGPT optimizes readability [5]. These systems do not evaluate a Coreutils whole-binary repair workflow driven by related official tests plus smoke checks. Our binary-level setting must reconstruct cross-function and global behavior rather than assume function-level context.

Evaluation and repair feedback. Liu et al. [10] make the readability, recompilability, and functionality split explicit; Decompile-Bench broadens decompilation measurement, and BinMetric situates code understanding within a broader binary-analysis benchmark suite [14, 17]. Equivalence checking, type recovery, and decompiler fidelity studies identify why recovered C can remain semantically fragile [3, 4, 18]. In program repair, fact selection and repair agents show that test and runtime evidence can guide LLM edits [2, 13, 22]. This paper applies that feedback principle to decompiled C at binary granularity and uses related official tests not only as an endpoint but also as repair evidence.

7 Conclusion

Repairing decompiled C should not stop at recompilation. This paper presents a bounded workflow that uses compiler/linker diagnostics for build repair and test-gate feedback for semantic repair. On 104 Coreutils 9.5 binaries in the static-enriched track with available decompiler exports and deterministic exact-output smoke comparisons, 91 binaries recompile and pass the test gate. The result provides bounded repair evidence rather than a proof of equivalence: accepted binaries have recorded repair traces, and failures retain explicit stopping reasons. Future work will broaden benchmarks, add pseudo-only and component ablations, and improve root-cause analysis by failure family without relaxing the bounded repair policy.

References

- [1] Jordi Armengol-Estapé, Jackson Woodruff, Chris Cummins, and Michael F. P. O’Boyle. 2024. SLaDe: A Portable Small Language Model Decompiler for Optimized Assembly. In *Proceedings of the 2024 IEEE/ACM International Symposium on Code Generation and Optimization*. 67–80. doi:10.1109/CGO57630.2024.10444788
- [2] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *Proceedings of the 47th International Conference on Software Engineering*. 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [3] Luke Dramko, Jeremy Lacomis, Edward J. Schwartz, Bogdan Vasilescu, and Claire Le Goues. 2024. A Taxonomy of C Decompiler Fidelity Issues. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 379–396.
- [4] Luke Dramko, Claire Le Goues, and Edward J. Schwartz. 2025. Fast, Fine-Grained Equivalence Checking for Neural Decompilers. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3772368
- [5] Peiwei Hu, Ruigang Liang, and Kai Chen. 2024. DeGPT: Optimizing Decompiler Output with LLM. In *Proceedings of the 2024 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2024.24401
- [6] Linxi Jiang, Xin Jin, and Zhiqiang Lin. 2025. Beyond Classification: Inferring Function Names in Stripped Binaries via Domain Adapted LLMs. In *Proceedings of the 2025 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2025.240797
- [7] Nan Jiang, Chengxiao Wang, Kevin Liu, Xiangzhe Xu, Lin Tan, Xiangyu Zhang, and Petr Babkin. 2025. Nova: Generative Language Models for Assembly Code with Hierarchical Attention and Contrastive Learning. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=4ytRL3HJrq>
- [8] Gangyang Li, Xiuwei Shang, Shaoyin Cheng, Junqi Zhang, Li Hu, Xu Zhu, Weiming Zhang, and Nenghai Yu. 2025. PseudoFix: Refactoring Distorted Structures in Decompiled C Pseudocode. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*. 841–853. doi:10.1109/ASE63991.2025.00075
- [9] Puzhuo Liu, Peng Di, and Yu Jiang. 2025. Function renaming in reverse engineering of embedded device firmware with chatgpt. In *Proceedings of the 1st ACM SIGPLAN International Workshop on Language Models and Programming Languages*. 57–65. doi:10.1145/3759425.3763387
- [10] Puzhuo Liu, Yuhang Huang, Jianlei Chi, Peng Di, and Yu Jiang. 2026. CODEFUSE-DEBENCH: An Empirical Study on Readability, Recompilability, and Functionality. *arXiv preprint arXiv:2605.29490* (2026). doi:10.48550/arXiv.2605.29490
- [11] Puzhuo Liu, Chengnian Sun, Yaowen Zheng, Xuan Feng, Chuan Qin, Yuncheng Wang, Zhenyang Xu, Zhi Li, Peng Di, Yu Jiang, et al. 2025. Llm-powered static binary taint analysis. *ACM Transactions on Software Engineering and Methodology* 34, 3 (2025), 1–36. doi:10.1145/3711816
- [12] Puzhuo Liu, Yaowen Zheng, Chengnian Sun, Chuan Qin, Dongliang Fang, Mingdong Liu, and Limin Sun. 2023. Fits: Inferring intermediate taint sources for effective vulnerability analysis of iot device firmware. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 138–152. doi:10.1145/3623278.3624759
- [13] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl T. Barr, and Sergey Mechtaev. 2025. The Fact Selection Problem in LLM-Based Program Repair. In *Proceedings of the 47th International Conference on Software Engineering*. 2574–2586. doi:10.1109/ICSE55347.2025.00162
- [14] Xiuwei Shang, Guoqiang Chen, Shaoyin Cheng, Benlong Wu, Li Hu, Gangyang Li, Weiming Zhang, and Nenghai Yu. 2025. BinMetric: A Comprehensive Binary Code Analysis Benchmark for Large Language Models. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence*. 7715–7723. doi:10.24963/ijcai.2025/858
- [15] Hanzhuo Tan, Weihao Li, Xiaolong Tian, Siyi Wang, Jiaming Liu, Jing Li, and Yuqun Zhang. 2026. SK2Decompile: LLM-Based Two-Phase Binary Decompile-Compilation from Skeleton to Skin. In *The Fourteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=jSQFqdoidy>
- [16] Hanzhuo Tan, Qi Luo, Jing Li, and Yuqun Zhang. 2024. LLM4Decompile: Decompile Binary Code with Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 3473–3487. doi:10.18653/v1/2024.emnlp-main.203
- [17] Hanzhuo Tan, Xiaolong Tian, Hanrui Qi, Jiaming Liu, Siyi Wang, Zuchen Gao, Qi Luo, Jing Li, and Yuqun Zhang. 2025. Decompile-Bench: Million-Scale Binary-Source Function Pairs for Real-World Binary Decompile. In *Advances in Neural Information Processing Systems 38: Datasets and Benchmarks Track*. doi:10.52202/085713-0177
- [18] Yanzhong Wang, Ruigang Liang, Yilin Li, Peiwei Hu, Kai Chen, and Bolun Zhang. 2025. TypeForge: Synthesizing and Selecting Best-Fit Composite Data Types for Stripped Binaries. In *Proceedings of the 2025 IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2847–2864. doi:10.1109/SP61157.2025.00193
- [19] Wai Kin Wong, Daoyuan Wu, Huaijin Wang, Zongjie Li, Zhibo Liu, Shuai Wang, Qiyi Tang, Sen Nie, and Shi Wu. 2025. DecLLM: LLM-Augmented Recompilable Decompile for Enabling Programmatic Use of Decompiled Code. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1841–1864. doi:10.1145/3728958
- [20] Danning Xie, Zhuo Zhang, Nan Jiang, Xiangzhe Xu, Lin Tan, and Xiangyu Zhang. 2024. ReSym: Harnessing LLMs to Recover Variable and Data Structure Symbols from Stripped Binaries. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4554–4568. doi:10.1145/3658644.3670340
- [21] Xiangzhe Xu, Zhuo Zhang, Zian Su, Ziyang Huang, Shiwei Feng, Yapeng Ye, Nan Jiang, Danning Xie, Siyuan Cheng, Lin Tan, and Xiangyu Zhang. 2025. Unleashing the Power of Generative Model in Recovering Variable Names from Stripped Binary. In *Proceedings of the 2025 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2025.240276
- [22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R. Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37*. <https://openreview.net/forum?id=mXpq6ut8J3>
- [23] Zhiping Zhou, Xiaohong Li, Ruitao Feng, Yao Zhang, Yuekang Li, Wenbu Feng, Yunqian Wang, and Yuqing Li. 2026. FidelityGPT: Correcting Decompile Distortions with Retrieval Augmented Generation. In *Proceedings of the 2026 Network and Distributed System Security Symposium*. doi:10.14722/ndss.2026.230989